

Ideal Hermes Setup

Skills · Rules · SOUL · Memory — a promptable blueprint

For: Nous Research Hermes Agent

Edition: 2026-07-23 · **Schema:** hermes.ideal-guidance/v1

How to use: Send this PDF. Paste **58 Master Prompt** into Hermes. Review staged writes. Done.

Stance: Thin always-on policy. Fat procedures as on-demand skills. Never dump the library into every turn.

0. One sentence

Ship an agent that is **safe by default, capable on demand, and cheap every turn** — using Hermes-native files (SOUL.md, AGENTS.md, ~/.hermes/skills/, memory, config.yaml) the way progressive-disclosure harnesses intend.

1. Why this layout (high confidence)

Failure mode	Fix in Hermes
Fat always-on instruction files tax every turn	Keep SOUL + AGENTS short; put procedures in skills
Context dumps / paste thrash	Prefer skills + MEMORY.md ; never paste prior chats into prompts
Self-certify “looks good”	Skill: verify-before-done — evidence before claims
Irreversible data loss	Skill: data-loss-stop — halt + explicit consent
Agent invents wrong “memory”	memory.write_approval: true until trusted
Agent mutates its own playbooks badly	skills.write_approval: true until trusted
Project rules leak into global persona	SOUL = everywhere; AGENTS = this repo only

Equation:

Quality ≈ model × harness × human process
Harness here = file placement + progressive skill load + write gates

2. Authority map (what goes where)

Layer	Path	Always on?	Token budget target	Owns
Identity	~/.hermes/SOUL.md	Yes (slot #1)	≤ ~400–600 tokens	Tone, safety invariants, pushback
User profile	~/.hermes/memories/USER.md	Snapshot	≤ ~500 tokens	Your prefs (agent-managed)
Agent notes	~/.hermes/memories/MEMORY.md	Snapshot	≤ ~800 tokens	Cross-session facts worth keeping
Project contract	./AGENTS.md (or .hermes.md)	Yes (one project file)	≤ ~600–800 tokens	Repo golden path, denies, verify pointer
Skills catalog	Hermes skills list	Catalog only	~few k	Names + descriptions
Skill body	~/.hermes/skills/**/SKILL.md	On demand (skill_view)	When needed	Procedures, pitfalls, verify steps
Config gates	~/.hermes/config.yaml	Runtime	n/a	write_approval, toolsets

Priority for project context (first match wins): .hermes.md / HERMES.md → AGENTS.md → CLAUDE.md → .cursorrules. Prefer **one** of these — usually AGENTS.md.

Golden rule: If it must apply *everywhere*, put it in SOUL.md. If it belongs to *one project*, put it in AGENTS.md. If it is a *procedure*, make it a **skill**.

3. Ideal directory tree

```
~/hermes/
├── SOUL.md # Global identity (thin)
├── config.yaml # write_approval + tools
├── memories/
│   ├── MEMORY.md # Agent notes (thrifty)
│   └── USER.md # User prefs (thrifty)
├── skills/
│   ├── safety/
│   │   └── data-loss-stop/SKILL.md
│   ├── engineering/
│   │   ├── verify-before-done/SKILL.md
│   │   ├── git-commit-safe/SKILL.md
│   │   └── create-pr-safe/SKILL.md
│   ├── research/
│   │   └── research-with-sources/SKILL.md
│   └── meta/
│       └── skill-authoring/SKILL.md # How Hermes writes better skills
└── <your-repo>/
    └── AGENTS.md # Project contract (thin)
```

Optional later: category folders (dev/, ops/), Hub installs, project-local skills via `skills.external_dirs` in config.

4. Ideal SOUL.md (global — copy as target)

Keep this **short**. No project paths. No 40-step runbooks.

Soul

You are a capable, direct operator. Prefer evidence over reassurance.

Standing rules

- Be concise. Lead with the answer. Bold sparingly.
- Push back when the user is wrong or the request is unsafe/underspecified.
- Never claim “done” without running the project’s verify path (see AGENTS.md / verify-before-done skill
- Never read, quote, or commit secrets (`.env`, `\*.pem`, `\*.key`, `\*credentials\*.json`).
- Before irreversible data loss (DROP/TRUNCATE/broad DELETE, prod deletes, destroy), STOP – load skill
- Prefer tools and skills over guessing. Prefer small diffs over rewrites.
- Do not dump large context into memory. Memory is for durable, reusable facts only.

Anti-habits

- No sycophancy (“great idea!” as a substitute for checking).
- No blank retry loops – terminalize failure with a next action.
- No inventing private ciphers or undocumented shorthand as “compression.”

5. Ideal AGENTS.md (per project — template)

Customize per repo. Stay under ~800 tokens.

AGENTS.md – <Project Name>

Product / goal

One sentence: what this repo is for.

Golden path

- Root: <path>
- Build: <cmd>
- Lint: <cmd>
- Test: <cmd>
- Verify-as-done: <cmd or checklist>

Authority

1. Human direction + law/security
2. CI / tests / schemas
3. This AGENTS.md
4. Skills on demand
5. Chat memory (lowest)

Hard denials

- Never commit secrets
- Never force-push main/master unless explicitly asked
- Never skip hooks unless explicitly asked
- One writer at a time on a shared checkout when multi-agent

When stuck

1. Reproduce with the verify command
 2. Load the relevant skill (verify-before-done, git-commit-safe, ...)
 3. Ask ≤3 clarifying questions only if goal/scope is ambiguous
-

6. Ideal config.yaml gates

Add or merge into ~/.hermes/config.yaml:

```
skills:
  write_approval: true    # stage skill creates/edits until you trust the loop
  # guard_agent_created: true # optional extra guard

memory:
  memory_enabled: true
  user_profile_enabled: true
  memory_char_limit: 2200
  user_char_limit: 1375
  write_approval: true    # review memory writes early on

display:
  memory_notifications: on
```

Review with:

- /skills pending · /skills diff <name> · /skills approve <name>
- /memory pending · /memory approve / reject

After a week of good behavior, you may set either gate to false. Keep them on for shared or production-adjacent machines.

7. Ideal starter skill pack (create these)

Each skill = folder + SKILL.md (agentskills.io / Hermes format). **Descriptions must include WHAT + WHEN** (third person) — the catalog uses them for discovery.

7.1 data-loss-stop

```
---
name: data-loss-stop
description: >-
  Halt before irreversible data loss and require explicit user consent.
  Use for SQL DROP/TRUNCATE/broad DELETE, production storage deletes,
  destroying cloud projects/resources, wiping secrets or KMS keys.
version: 1.0.0
metadata:
  hermes:
    tags: [Safety, Ops, Consent]
---
# Data Loss Stop

## When to Use
Any command that permanently destroys data or production resources.

## Procedure
1. Halt — do not execute.
2. State impact, blast radius, and why it seems necessary.
3. Ask for explicit affirmative consent in-thread.
4. Only then run the command.
5. Report what was destroyed and how to verify.

## Pitfalls
- “Probably a staging DB” without proof is not consent.
- Broad DELETE without WHERE is always high-risk.

## Verification
Confirm with a read-only check that the intended objects are gone and nothing else was harmed.
```

7.2 verify-before-done

```
---
name: verify-before-done
description: >-
  Run the project verify path and report evidence before claiming work is done.
  Use when finishing a task, before PR, or when the user asks if something works.
version: 1.0.0
metadata:
  hermes:
    tags: [Engineering, Verify, Quality]
    related_skills: [git-commit-safe, create-pr-safe]
---
# Verify Before Done

## When to Use
Before saying done, shipping, or opening a PR.
```

Procedure

1. Read AGENTS.md verify / golden-path commands.
2. Run lint → tests → project verify (adapt to stack).
3. For each gate: pass/fail + command + first actionable error lines.
4. Only claim done if required gates pass (or user waived in writing).

Pitfalls

- “Looks good” / “should work” without sensors is forbidden.
- Skipping failing tests without user approval is forbidden.

Verification

Paste the verify command outputs (trimmed) as evidence.

7.3 git-commit-safe

```
---
name: git-commit-safe
description: >-
  Create a safe git commit only when the user asks. Use when committing,
  staging, or drafting a commit message. Never amend/force/skip hooks unless asked.
version: 1.0.0
metadata:
  hermes:
    tags: [Git, Engineering]
---
```

Git Commit Safe

When to Use

User explicitly asks to commit.

Procedure

1. `git status` · `git diff` (staged+unstaged) · `git log -5 --oneline` in parallel.
2. Do not commit secrets (`.env`, credentials, pem/key).
3. Stage only relevant files.
4. Commit with a concise why-focused message (1-2 sentences).
5. `git status` to confirm. Do not push unless asked.

Pitfalls

- No `git commit --amend` unless user asked and commit is yours/unpushed.
- No `--no-verify` unless user asked.

7.4 create-pr-safe

```
---
name: create-pr-safe
description: >-
  Open a GitHub pull request with gh after analyzing branch vs base.
  Use when the user asks to create a PR or pull request.
version: 1.0.0
metadata:
  hermes:
    tags: [Git, GitHub, Engineering]
    related_skills: [git-commit-safe, verify-before-done]
---
```

```
# Create PR Safe
```

Procedure

1. status · diff · tracking · log vs base · `git diff base...HEAD`.
2. Push `-u` if needed.
3. `gh pr create` with Summary (1-3 bullets) + Test plan checklist.
4. Return the PR URL.

Pitfalls

- Never force-push main/master.
- Include ALL commits on the branch, not only the latest.

7.5 research-with-sources

```
---
name: research-with-sources
description: >-
  Research with primary sources, confidence labels, and citations.
  Use for deep dives, market/tech research, or when the user asks for proofs.
version: 1.0.0
metadata:
  hermes:
    tags: [Research, Sources]
    requires_toolsets: [web]
---
# Research With Sources
```

Procedure

1. Define the question and success criteria.
2. Prefer primary sources (docs, papers, vendor pages) over blogs.
3. For each claim: statement · confidence (high/med) · source URL/title.
4. Separate facts from estimates.
5. End with a short actionable recommendation.

Pitfalls

- Do not invent citations. If unverified, say so.

7.6 skill-authoring

```
---
name: skill-authoring
description: >-
  Create or update Hermes skills with correct frontmatter, progressive
  disclosure, and WHAT+WHEN descriptions. Use when writing SKILL.md files.
version: 1.0.0
metadata:
  hermes:
    tags: [Meta, Skills]
---
# Skill Authoring
```

Rules

- One skill = one job.
- Description: third person, WHAT + WHEN, trigger keywords.
- Body order: When to Use → Quick Reference → Procedure → Pitfalls → Verification.

- Put edge cases at the bottom (progressive disclosure).
- Prefer `stdlib/curl/existing tools; scripts/` for parsers.
- Never put secrets in skills; use `required_environment_variables`.

8. Master Prompt — paste into Hermes

Copy everything in the box below into a Hermes chat (CLI or gateway). Hermes should create the files; with `write_approval: true`, approve via `/skills` and file writes as prompted.

```
You are setting up the Ideal Hermes guidance layout for this machine / project.

GOAL
Create a thin always-on policy + on-demand skills pack. Do not dump procedures into SOUL or AGENTS.

DO THIS IN ORDER
1) Show me your plan (file tree + what goes in each file). Wait for my OK if anything is destructive.
2) Ensure ~/.hermes exists. Create/update:
  - ~/.hermes/SOUL.md (use the Ideal SOUL from the blueprint – short)
  - Merge into ~/.hermes/config.yaml:
    skills.write_approval: true
    memory.write_approval: true
    memory.memory_enabled: true
    memory.user_profile_enabled: true
3) Create these skills under ~/.hermes/skills/ (SKILL.md each, Hermes frontmatter):
  - safety/data-loss-stop
  - engineering/verify-before-done
  - engineering/git-commit-safe
  - engineering/create-pr-safe
  - research/research-with-sources
  - meta/skill-authoring
  Use the procedures from the Ideal Hermes Setup blueprint. Descriptions MUST include WHAT + WHEN.
4) In the current project root, create or refresh AGENTS.md from the Ideal template.
  Fill Golden path / Build / Lint / Test / Verify from this repo if detectable; otherwise leave TODOs.
5) Do NOT invent long MEMORY.md content. Leave memory for real sessions.
6) Print a verification checklist:
  - SOUL token-ish length (keep short)
  - AGENTS present
  - skills_list shows the six skills
  - config gates on
7) Stop. Summarize paths created and any approvals I still need (/skills pending, etc.).

CONSTRAINTS
- Prefer patching existing files over wiping them; if SOUL.md already has custom voice, merge standing r
- No secrets in any file.
- One job per skill. No mega-skills.
```

9. Day-2 operating habits

Habit	Do this
New workflow learned	Ask Hermes: "Save that as a skill" (then <code>/skills approve</code>)
New repo	Drop a thin AGENTS.md first; don't bloat SOUL
Model swap mid-task	Avoid; finish the task or start a clean session with note
"Make it better"	Add acceptance criteria first (≤ 3 clarifies)

Claiming done	Trigger verify-before-done
Dangerous ops	Expect data-loss-stop

10. What NOT to build

Anti-pattern	Why
5,000-token SOUL.md	Fixed tax every turn
Duplicating SOUL into every AGENTS.md	Drift + confusion
One “god skill” for everything	Discovery fails; tokens explode on load
Putting API keys in SKILL.md	Use <code>required_environment_variables</code> / <code>.env</code>
Treating MEMORY.md as a wiki	Char limits exist; use skills/docs instead
Skipping verify because “simple change”	Self-certify is the #1 false-green path

11. Success criteria (you’re done when)

- ☐ SOUL.md is short and global-only
- ☐ Each repo has one thin AGENTS.md (Or .hermes.md)
- ☐ Six starter skills appear in `skills_list` / slash commands
- ☐ `skills.write_approval` and `memory.write_approval` are on (until trusted)
- ☐ A sample task uses `verify-before-done` before “done”
- ☐ A dangerous command triggers `data-loss-stop` behavior

12. Source notes

Grounded in Hermes docs (SOUL / context files / skills progressive disclosure / `write_approval`) and 2025–2026 harness practice: **rules always-on thin · skills on demand · verify outside the generator · consent before irreversible loss**. Compatible with agentskills.io.

Send this PDF → paste §8 → approve staged writes → customize AGENTS.md per repo.